

MVC (Model-View-Controller)

What is Model Interface ?

- The Model interface in Spring MVC is used to store data.
- It acts as a container to pass data from the Controller to the View.
- The data is stored in key-value pairs.
- It serves as a data carrier without containing any business logic.

Why Model Interface is Used?

- To pass data from Controller i.e. View
- To store multiple values as attributes
- To keep the presentation layer separate from the business logic
- To make the output dynamic (display values on JSP)

Example :-

```
model.addAttribute("name", "Sagar");
```

Here:

"name" → key

"Sagar" → value

This data can be shown in JSP using `${name}`.

Methods of Model Interface :-

Method	Description
<code>addAttribute(String name, Object value)</code>	Add an attribute with a key and value.
<code>addAttribute(Object value)</code>	Add an attribute with default key (class name).
<code>addAllAttributes(Map<String, ?> map)</code>	Add multiple attributes from a Map.
<code>mergeAttributes(Map<String, ?> map)</code>	Merge given attributes with existing ones.
<code>containsAttribute(String name)</code>	Check if an attribute exists.
<code>asMap()</code>	Get all attributes as a Map.

What is a Model ?

- The Model represents the data or state of the application.
- It is usually a POJO class with fields, constructors, getters, and setters.
- It does not contain UI logic.

What is a Controller ?

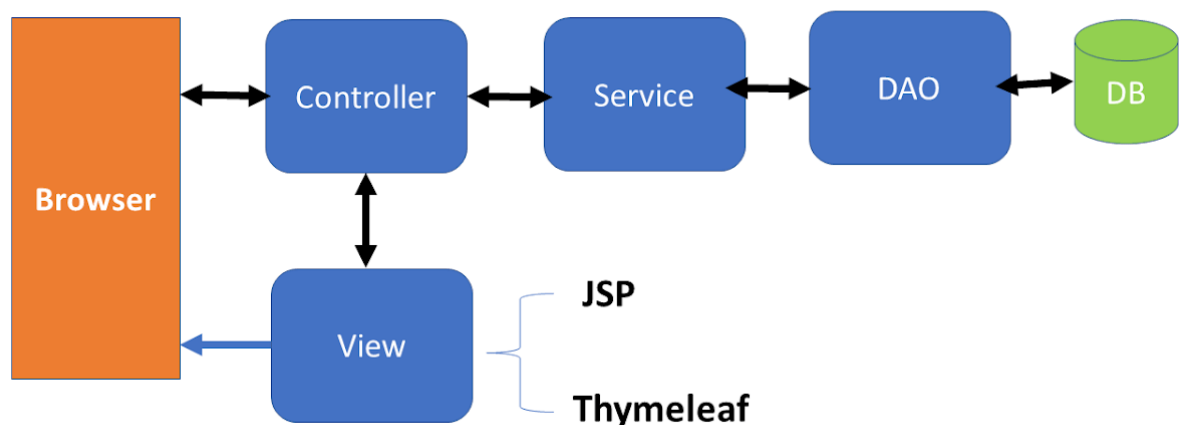
- The Controller handles user requests and decides what to do with them.
- It processes data and adds it to the Model.
- It returns the name of the View that will display the data.

What is a View ?

- The View is the UI layer (usually JSP, Thymeleaf, or HTML) that displays data to the user.
- It retrieves data from the Model using Expression Language (\${}).
- It contains no business logic, only presentation logic.

How Spring MVC Works -

- User sends a request to the application.
- DispatcherServlet directs the request to the appropriate Controller.
- Controller creates or retrieves Model data.
- Controller adds data to Model, ModelMap, or ModelAndView.
- Controller returns the View name.
- The View accesses the Model data using \${} and displays it.



Steps in Creating a Model -

Step 1: Create a POJO Class

- A Java class with private fields and getter/setter methods.
- Represents the data to be displayed or processed.
- Example fields: id, fullname, email, password, etc.

Step 2: Add Model Data in Controller

- Controller methods respond to user requests.
- These methods create objects of the Model (POJOs).
- The object is stored in the Model using:
 1. `model.addAttribute()`
 2. `modelMap.put()`
 3. Or returned using `ModelAndView`.

Step 3: Use the Model in the View

- The View (JSP) accesses the Model data using:
 1. `${attributeName.property}`
- The Model data is displayed on the webpage.

Implementation of Spring MVC Application :-

index.jsp -

```
<html>
<body>

    <!-- <form action="add" method="post">
        <input type="text" name="num1" placeholder="Enter number1"><br><br>
        <input type="text" name="num2" placeholder="Enter number2"><br><br>

        <button type="submit">Submit</button>
    </form> -->

    <form action="save" method="post">
        <input type="text" name="fullname" placeholder="Enter fullname"><br><br>
        <input type="email" name="email" placeholder="Enter email"><br><br>
        <input type="password" name="password" placeholder="Enter password"><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

pom.xml –

```
http://maven.apache.org/maven-v4_0_0.xsd (xsi:schemaLocation with catalog)
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.demo</groupId>
  <artifactId>Maven_Sample_Project2</artifactId>
  <packaging>war</packaging>
  <version>0.0.1-SNAPSHOT</version>
  <name>Maven_Sample_Project2 Maven Webapp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.springframework/spring-webmvc -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.32</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/javax.servlet/javax.servlet-api -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.springframework/spring-jdbc -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-jdbc</artifactId>
      <version>5.3.32</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.postgresql/postgresql -->
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.7.8</version>
    </dependency>
  </dependencies>
  <build>
    <finalName>Maven_Sample_Project2</finalName>
  </build>
</project>
```

spring-servlet.xml –

```
http://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation) | http://www
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:mvc="http://www.springframework.org/schema/mvc"
       xmlns:context="http://www.springframework.org/schema/context"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans.xsd
         http://www.springframework.org/schema/context
         http://www.springframework.org/schema/context/spring-context.xsd
         http://www.springframework.org/schema/mvc
         http://www.springframework.org/schema/mvc/spring-mvc.xsd">

    <context:component-scan base-package="com.demo.controllers"></context:component-scan>

    <!-- View Resolver -->
    <bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
        <property name="prefix" value="/WEB-INF/pages/" />
        <property name="suffix" value=".jsp" />
    </bean>

    <bean id="dataSource" class="org.springframework.jdbc.datasource.DriverManagerDataSource">
        <property name="driverClassName" value="org.postgresql.Driver" />
        <property name="url" value="jdbc:postgresql://localhost:5433/springmaven1" />
        <property name="username" value="postgres" />
        <property name="password" value="4518" />
    </bean>

    <bean id="jdbcTemplate" class="org.springframework.jdbc.core.JdbcTemplate">
        <property name="dataSource" ref="dataSource" />
    </bean>

    <!-- Enable Spring MVC -->
    <mvc:annotation-driven />

</beans>
```

Student.java – i.e. Model

```
package com.demo.models;

public class Student {
    private int id;
    private String fullname;
    private String email;
    private String password;

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getFullname() {
        return fullname;
    }
    public void setFullname(String fullname) {
        this.fullname = fullname;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getPassword() {
        return password;
    }
    public void setPassword(String password) {
        this.password = password;
    }
}
```